

PRD + Information Architecture: Exposure-Aware Fleet Routing (Delhi NCR)

2026-09-19 · @Someone

1. Overview

This product routes and assigns delivery riders in Delhi NCR by time **and** pollution exposure, so delivery windows are still met while each rider's cumulative PM2.5 dose drops and is spread fairly across the fleet.

Problem. During winter months, air quality in Delhi NCR heavily impacts logistics and delivery services. Companies need to dynamically route their fleets not just on traffic, but to avoid sending unmasked riders into micro-zones with severe AQI spikes, while still hitting their delivery windows.

Solution. Google Routes API generates traffic-aware candidate routes (Option A, hybrid). Neo4j scores each candidate by exposure, using a zone-level PM2.5 layer built from historical sensor CSVs and replayed as a live timeline. The engine picks the lowest-exposure route that still meets the window, explains the choice with Gemini, and pushes alerts to riders on the web.

Why Neo4j. Sensors, zones, routes, riders, orders, place aliases and policy rules are connected data. One graph answers "which zones does this route cross, what was the air like when the rider got there, which rules apply to this vehicle, and what did this rider already breathe today" without joins across systems.

2. Goals, non-goals and locked decisions

The MVP must show one clear result: a lower PM2.5 dose for the same delivery window, on a replayed real smog episode.

Goals

1. Cut per-route PM2.5 dose against the fastest route while meeting the delivery window.
2. Spread cumulative exposure fairly across riders, not just minimize the total.
3. Resolve informal place searches ("cp delhi") into a correct, geocodable place.
4. Make every routing decision explainable and traceable to sensor data and rules.

5. Alert riders and dispatchers on the web when conditions change.
6. Be demoable end to end in replay mode without depending on live data.

Non-goals (MVP)

- Own road-network routing (OSM + Neo4j GDS). Stretch goal only.
- Live AQI or weather API data. Deferred; the schema carries `source` and `confidence` so it can be added later.
- ML forecasting of PM2.5. Stretch goal, and only if it beats a persistence baseline.
- Native mobile apps, payments, real order-system integrations, multi-tenant accounts.
- Health or medical claims. The product reports exposure reduction, not health outcomes.

Locked decisions

Area	Decision	Consequence
Routing	Option A, hybrid: Google generates candidates, Neo4j scores them	No GDS dependency; real traffic times; only 1 to 3 alternatives per request
Pollution data	Subset of historical sensor CSVs (Nov 2020), replayed as a timeline	Demo is a replay, labeled as such. Live API fallback comes later
Frontend	Streamlit dispatcher console	Reruns on interaction, so state must live in session state and Neo4j
Backend	Python (shared core package, FastAPI for rider app, push and clock)	One scoring codebase used by both UIs
Database	Neo4j (AuraDB)	Load aggregates only, never raw 2.5-second rows
AI	Gemini for place normalization and grounded explanations	Never the source of truth for coordinates
Maps	Google Geocoding, Routes API, Maps JavaScript API	Routes displayed on a Google map
Hosting	Render	Free tier sleeps; wake before demos

Area	Decision	Consequence
Alerts	Web push	Needs a service worker; see the notifications section
Knowledge graph	Proposed: place aliases, GRAP policy rules, grounded explanations	Not yet confirmed by the team

3. Users and scenarios

The primary user is the dispatcher who plans and monitors deliveries; riders are the second audience and receive alerts.

Persona	Needs	Uses
Dispatcher / fleet ops manager (primary)	Meet delivery windows while limiting rider exposure; justify decisions	Streamlit console: route planner, fleet map, alerts, policy
Rider (secondary)	Clear next step, warning before entering bad air, a fair share of hard zones	Rider web app: current order, route, exposure meter, push alerts
Fleet / ESG lead (secondary)	Evidence of exposure reduction and fairness	Reports view: exposure by rider and zone, before vs after

Key scenarios

- 1. Plan one delivery.** Dispatcher types "cp delhi". The place resolves to Connaught Place, three candidate routes appear with time, dose and window margin, and the lowest-dose route that meets the window is recommended.
- 2. Spike mid-shift.** A zone on an active route crosses the alert threshold. The engine proposes a reroute, the rider gets a push notification, and the dispatcher sees the trade-off (extra minutes vs dose saved).
- 3. Budget guard.** A rider reaches 80% of the shift exposure budget. The next order goes to another eligible rider, and the dispatcher is told why.
- 4. Policy change.** The GRAP stage is raised. Vehicle types restricted in affected zones drop out of the candidate list, and routes are re-scored.

5. **End-of-shift review.** The ESG lead sees exposure by rider, how evenly it was spread, and the dose avoided against fastest-route baselines.

4. Scope and release plan

The MVP proves one loop: search, candidate routes, exposure score, recommendation, spike alert. Everything else builds on that loop, so cut from the bottom of this table first.

Release	Contents	Cut order
MVP (P0)	CSV cleaning and 15-minute aggregation; zone layer in Neo4j; Gemini place search with alias cache; Google candidate routes; dose scoring and window filter; replay clock with spike injector; dispatcher map and route comparison; Gemini explanation; push alert for spike and reroute	Last to cut
P1	Rider exposure budgets and assignment; fairness rotation; GRAP policy rules; via-waypoint avoidance routes; rider web app with exposure meter; exposure ledger and reports	Cut second
P2 / later	Live AQI API fallback for uncovered areas; short-horizon PM2.5 forecast; own routing on OSM with GDS; sensor reliability scoring; multi-city	Cut first

Rules of scope

- The demo area is limited to the neighborhoods the chosen sensors cover. Extending it needs the API fallback, which is out of MVP.
- Push notifications are in the MVP for spike and reroute alerts only. If the service worker setup slips, the fallback is in-app alerts in both UIs.
- A single composite "safety score" is not built. Time, dose and window margin are always shown separately.

5. Functional requirements

Twenty-four requirements across eight areas; P0 items form the MVP loop.

ID	Requirement	Priority	Acceptance criteria
DT-1	Ingest the selected CSV files, clean them and aggregate to 15-minute medians per device	P0	Output has one row per device and bucket; drop counts reported per cleaning rule
DT-2	Assign sensors and buckets to H3 zones and load zone readings into Neo4j with <code>source</code> and <code>confidence</code>	P0	Every zone reading carries source, confidence and sensor count
DT-3	Replay clock with speed control (1x to 600x) and jump-to-time	P0	Changing the clock changes every AQI value shown in the UI
DT-4	Spike injector that raises PM2.5 in a chosen zone for a set duration	P0	Injected value is labeled as simulated in the UI
PL-1	Place search resolves an informal query to a place and coordinates	P0	Alias cache checked first, then Gemini normalization, then geocoder
PL-2	Ambiguous or low-confidence queries show up to 3 candidates to pick from	P0	No silent guess when confidence is below the threshold (default 0.7)
PL-3	Confirmed resolutions are written back to the alias graph	P1	Second search for the same alias skips Gemini
RT-1	Request traffic-aware alternatives from Google Routes for start, destination and departure time	P0	1 to 3 routes returned with duration, distance and polyline
RT-2	Decode each polyline, map it to H3 zones with time-in-zone	P0	Route stores an ordered zone list with seconds per zone
RT-3	Score each route: total dose, ETA, window margin, coverage percent	P0	Scores use the reading for the time the rider will be in each zone
RT-4	Recommend the lowest-dose route that meets the window, else fastest plus a "window at risk" flag	P0	Recommendation shows delta vs fastest in dose percent and minutes
RT-5	Avoidance routes via waypoints when every alternative crosses a severe zone	P1	At most 2 extra Google requests per order

ID	Requirement	Priority	Acceptance criteria
EX-1	Dose calculation with configurable rider activity factor and mask type	P0	Defaults documented; "unmasked" is the baseline
EX-2	Per-rider shift exposure budget and running total	P1	Meter updates as the replay clock advances
DP-1	Candidate rider query: within reach, budget left, vehicle not restricted	P1	Ineligible riders show the reason
DP-2	Fairness-aware assignment that avoids repeatedly sending the same rider into severe zones	P1	Exposure spread beats a nearest-rider baseline in the simulation
KG-1	Alias graph of NCR places (landmarks, metro stations, markets, sectors)	P0	At least 150 seeded aliases, each linked to a place and a zone
KG-2	GRAP stage and rule graph restricting vehicle types by zone	P1	Changing the active stage changes candidate riders and route flags
KG-3	Grounded explanation: Gemini writes one sentence from a retrieved subgraph	P0	Every number in the sentence comes from the subgraph passed in
NT-1	Push notification to riders on spike and reroute	P0	Delivered while the rider page is closed but the browser is running
NT-2	Dispatcher in-app alert feed with acknowledge	P0	Alert stored in Neo4j with status and timestamps
UI-1	Dispatcher console: replay bar, fleet map, KPIs, alerts	P0	All views follow the replay clock
UI-2	Route comparison view with side-by-side cards and Google map	P0	Shows time, dose, margin and confidence for each route
UI-3	Data and method page: sources, cleaning rules, confidence definitions	P1	Reachable from every screen

6. Exposure model and routing logic

Route dose is the PM2.5 concentration in each zone, multiplied by the time the rider spends there, summed along the route. It uses concentration in $\mu\text{g}/\text{m}^3$, not the AQI index, because AQI is not linear in concentration.

$$D_{\text{route}} = \sum_{i=1}^n C_{\text{z}_i}(t_i) \cdot \Delta t_i \cdot A \cdot (1 - F_{\text{mask}})$$

- C is the zone PM2.5 ($\mu\text{g}/\text{m}^3$) at the time t_i the rider reaches zone i , not at departure.
- Δt is time spent in the zone, in hours, from the Google route duration.
- A is a rider activity factor (breathing-rate multiplier). Default 1.0, configurable.
- F_{mask} is mask filtration, 0 for the "unmasked" baseline. Mask types are configurable values, not medical claims.
- Unit is $\mu\text{g}\cdot\text{h}/\text{m}^3$ (relative dose). The shift budget uses the same unit.

Zone PM2.5 from sensors

$$C_z = \frac{\sum_j w_j \cdot p_j}{\sum_j w_j}, \quad \text{quad } w_j = \frac{1}{d_j^2}, \quad \text{quad } d_j \leq R$$

Sensors j within radius R (default 3 km) of the zone centroid are weighted by inverse squared distance.

Decision rule for one order

1. Request up to 3 traffic-aware alternatives from Google Routes.
2. For each, compute duration, ETA, window margin ($\text{window_end} - \text{ETA}$), dose and coverage percent.
3. Discard routes with margin below the safety buffer (default 2 minutes).
4. Recommend the lowest-dose remaining route; break ties by shorter duration.
5. If none qualifies, recommend the fastest route, flag "window at risk", and alert the dispatcher.
6. Always show the delta against the fastest route: dose percent and minutes.

Severity bands for display and alerts

Band	PM2.5 ($\mu\text{g}/\text{m}^3$)	Use
Good	0 to 30	Map color only
Satisfactory	31 to 60	Map color only
Moderate	61 to 90	Map color only
Poor	91 to 120	Map color only
Very poor	121 to 250	Alert threshold (default)
Severe	above 250	Alert threshold; avoidance routing

These follow the CPCB PM2.5 breakpoints as I recall them; confirm against the current CPCB table before publishing. CPCB bands are defined on 24-hour averages, so here they are display heuristics applied to 15-minute medians.

Design rules

- Show dose, time and margin as separate numbers. No blended safety score.
- Every score shows coverage percent. Below 80% it is marked low confidence.
- Copy says "exposure reduction", never "health protection".
- Google traffic durations are real-time values from the day of the demo, while pollution is the replayed 2020 data. Say so on the data and method page.

7. Web push notifications

Riders get web push alerts from a small rider web app served by the FastAPI service, because a Streamlit page cannot register its own service worker. Dispatchers get an in-app alert feed inside Streamlit.

Triggers

Trigger	Recipient	Message intent	Action on tap
A zone ahead on the active route crosses the alert threshold	Rider, dispatcher	"High PM2.5 ahead on your route. Reroute available."	Open rider order view with new route
Reroute accepted or auto-applied	Rider	New route, extra minutes, dose saved	Open order view

Trigger	Recipient	Message intent	Action on tap
Shift exposure budget reaches 80%	Rider, dispatcher	Budget warning	Open exposure meter
Order window at risk after re-scoring	Dispatcher	Window margin below buffer	Open order in console
GRAP stage changed	Dispatcher	Vehicle restrictions updated	Open policy page

Flow

```
sequenceDiagram
    participant T as Replay tick
    participant E as Alert evaluator
    participant N as Neo4j
    participant P as Push service
    participant S as Service worker
    participant R as Rider device
    T->>E: clock advances
    E->>N: read zone readings on active routes
    E->>N: write Alert node
    E->>P: send(alert, rider subscription)
    P->>S: Web Push message
    S->>R: show notification
    R->>N: tap opens order view, alert acknowledged
```

The evaluator runs on every replay tick and skips an alert if the same rider and zone already alerted within 15 minutes of replay time.

Technical constraints

- Web Push needs HTTPS (Render provides it), VAPID keys, a service worker file served from the site root, and explicit user permission.
- The service worker and the permission prompt live in the rider app, not in a Streamlit iframe component.
- Subscriptions are stored per rider in Neo4j (`PushSub` node). Treat endpoints and keys as secrets.
- On iPhone, Web Push works only when the site is added to the Home Screen (iOS 16.4 or later, as far as I know). Confirm before promising it in the demo.

- Fallback if push slips: the same alerts appear as in-app toasts in both UIs, since they are already stored as `Alert` nodes.

8. Success metrics

The demo headline is one line per order: **"-X% PM2.5 dose, +Y min, window met"**, measured against the fastest Google route. The targets below are proposals; recalibrate after the CSV profiling run shows how much spatial variation the data has.

Metric	Definition	Proposed target
Dose reduction	Median percent dose saved vs the fastest route, across replay orders	15% or more
Time cost	Median extra minutes vs the fastest route	10% of fastest ETA or less
Window compliance	Orders delivered inside window in the simulation	95% or more
Exposure spread	Gini coefficient of rider exposure at shift end	Lower than the nearest-rider baseline
Place resolution	Top-1 accuracy on at least 50 labeled NCR queries	90% or more
Scoring latency	Time from search to ranked routes, including the Google call	Under 5 s at p95
Alert delivery	Push alerts shown within 10 s of the tick that raised them	95% or more

Baselines. Fastest Google route for routing, nearest available rider for assignment. Both run in the same replay so comparisons are like for like.

Evaluation set. Log every search, resolved place and user correction from the first day. That log becomes the labeled set for place resolution accuracy.

9. Risks, assumptions and open questions

The biggest unknown is sensor coverage, because the profiling run on the CSVs has not happened yet.

Risk	Impact	Mitigation
Sensor coverage is sparse or clustered	Route scores are guesses away from sensors	Limit the demo area to covered zones; show coverage percent; mark low confidence below 80%
Live traffic vs replayed 2020 pollution	Time and dose come from different days	Use Google only for duration and geometry; disclose on the data and method page
Low-cost sensors drift and react to humidity	Inflated or noisy PM2.5	Cleaning rules, 15-minute medians, confidence flags
Google returns few alternatives	Little room to reduce dose	Via-waypoint avoidance routes (P1), capped at 2 extra requests
Google Maps terms limit caching and non-Google display of route data	Rejected or non-compliant build	Show routes on a Google map; store only own derived scores and zone lists; re-check the current terms
Streamlit cannot host a service worker	Push does not work	Rider web app served from FastAPI; in-app alert fallback
Render free tier sleeps; disk is ephemeral	Cold start in front of judges	Wake before demo; keep state in Neo4j; no local files
AuraDB Free limits and inactivity pause	Load failures or a paused database	Load aggregates only; check current limits and keep a resume routine
Gemini invents or misreads a place	Wrong endpoint, wrong route	Geocoder is the source of truth; confidence threshold; candidate picker
Dose is an approximation	Over-claiming	Say "exposure reduction"; document the formula and defaults
GRAP rules change over time	Stale policy graph	Hand-curate from official notices with a source and effective date on each rule

Assumptions

- The provided CSV subset covers the demo area and includes at least one clear pollution rise.
- Orders, riders and hubs are simulated; no real fleet integration.
- CSV timestamps are in a single known time zone (assumed IST until confirmed).

Open questions

- ☐ How many devices are in the chosen files, where are they, and are they fixed or mobile? (Run the profiling script.)
- ☐ What date range does the subset cover, and does it contain a spike worth replaying?
- ☐ Which districts form the demo area?
- ☐ How many riders, orders and hubs does the simulation use?
- ☐ Is the delivery window a per-order slot (for example 30 minutes) or a per-shift rule?
- ☐ Do mask types ship in the MVP, or only the unmasked baseline?
- ☐ Is the knowledge graph scope (aliases, GRAP rules, grounded explanations) confirmed?
- ☐ Will riders receive push on real phones during the demo, or on a laptop browser?
- ☐ Team size and days available, to turn the build order into a task split.

10. System architecture

One shared Python core package does the scoring, and two thin front doors use it: the Streamlit dispatcher console and a FastAPI service that also serves the rider app and sends push.

flowchart LR

```

D[Dispatcher<br/>Streamlit console] --> C[Core package<br/>Python]
R[Rider web app<br/>PWA + push] --> A[FastAPI<br/>API, push, clock]
A --> C
C --> N[(Neo4j<br/>zones, riders, KG)]
C --> G[Google Maps<br/>Geocode + Routes]
C --> M[Gemini<br/>normalize + explain]
E[Offline ETL<br/>CSV to zone aggregates] --> N
A --> K[Replay clock<br/>+ alert evaluator]
K --> N

```

The Google and Gemini calls sit behind the core package, so both UIs share the same caching and keys.

Component	Responsibility	Notes
Streamlit console	Dispatcher screens, map embed, demo controls	Holds no business logic; state in <code>st.session_state</code> and Neo4j
FastAPI service	Rider app, push subscriptions, push sending, clock and evaluator, REST endpoints	Serves the service worker file from the site root
Core package	Place resolution, route scoring, dispatch, KG queries, explanations	Plain modules with tests; imported by both front doors
Neo4j	Zones, readings, sensors, riders, orders, routes, alerts, knowledge layer	AuraDB; aggregates only
Offline ETL	Clean and aggregate CSVs, build zones, seed aliases and rules	Run locally; loads results into Neo4j
Replay clock	Simulated time, speed, jump-to-time; drives the alert evaluator	One <code>DemoClock</code> node; ticks run inside the API process

Plan one delivery

```
sequenceDiagram
    participant U as Dispatcher
    participant C as Core
    participant N as Neo4j
    participant M as Gemini
    participant G as Google
    U->>C: search "cp delhi"
    C->>N: alias lookup
    C->>M: normalize (only if no alias hit)
    C->>G: geocode, bounded to NCR
    C->>G: routes with alternatives
    C->>N: zone readings for each route
    C->>C: dose, margin, pick route
    C->>M: explain from subgraph
    C->>U: ranked routes + explanation
```

11. Data pipeline and sources

Raw sensor rows never enter Neo4j. They are cleaned and reduced offline to 15-minute medians per device, then to zone readings, which is roughly a 350x reduction from a 2.5-second sampling interval.

Source data. Several CSV files of sensor readings from November 2020, one row about every 2.5 seconds. Only a subset is used.

Column	Meaning	Use
<code>uid</code>	Row id	Dropped
<code>dateTime</code>	Timestamp	Bucketing and replay clock
<code>deviceId</code>	Sensor id	Sensor node key
<code>lat</code> , <code>long</code>	Sensor position	Zone assignment; check whether it varies
<code>pm1_0</code> , <code>pm2_5</code> , <code>pm10</code>	Particulate concentration	<code>pm2_5</code> drives dose; <code>pm10</code> used for validity checks

Pipeline stages

Stage	What happens	Output
1. Profile	Device count, spread, time range, sampling gaps, coordinate variation, PM2.5 above PM10 share	Profile report; decides the file subset and demo area
2. Clean	Drop rows with PM2.5 above PM10, missing or zero coordinates, out-of-range values, flatlines and single-row jumps	Clean rows and a drop count per rule
3. Aggregate	15-minute median per device and bucket, with row count	Device-bucket table (Parquet or DuckDB)
4. Zone	Assign sensors to H3 zones (default resolution 8, about 0.74 km ² per hex)	Zone table with centroids and adjacency
5. Interpolate	Inverse-distance weighting per zone and bucket	Zone readings with source, confidence, sensor count

Stage	What happens	Output
6. Load	Write sensors, zones, buckets and zone readings to Neo4j	Graph ready for the replay
7. Seed	Load place aliases, hubs, simulated riders and orders, GRAP rules	Knowledge layer and fleet

Coverage rule per zone and bucket

Condition	Action	Confidence
2 or more sensors within R (default 3 km)	Inverse-distance weighting	High
1 sensor within R	Use it with distance decay	Medium
No sensor within R	No reading; zone excluded from the demo area in MVP	None

The API fallback for uncovered zones is deferred. When added, it writes readings with `source = model` and low confidence, so scoring code does not change.

Choosing the subset. Pick files for spatial spread across the demo area, a contiguous window that contains a clear pollution rise, and low share of invalid rows.

Replay clock. The clock maps demo time to a CSV timestamp. All queries read the zone reading for the clock's current bucket. Injected spikes are stored as separate readings with `source = simulated` and shown as such.

12. Neo4j graph model

One database holds two layers: an operational layer that scores routes and dispatches riders, and a knowledge layer that resolves places and applies policy rules. Zones join the two.

```

flowchart LR
    S[Sensor] -->|IN_ZONE| Z[Zone]
    ZR[ZoneReading] -->|OF| Z
    ZR -->|DERIVED_FROM| S
    RT[Route] -->|PASSES_THROUGH| Z
    O[Order] -->|HAS_CANDIDATE| RT
    RD[Rider] -->|ASSIGNED| O
    RD -->|EXPOSED_IN| Z
    AL[Alert] -->|ABOUT| O
    A[Alias] -->|REFERS_TO| P[Place]
    P -->|IN_ZONE| Z
    GS[GRAPStage] -->|TRIGGERS| RU[Rule]
    RU -->|RESTRICTS| V[VehicleType]
    RU -->|APPLIES_TO| Z

```

The top half is the operational layer; `Alias`, `Place`, `GRAPStage`, `Rule` and `VehicleType` are the knowledge layer.

Node labels

Label	Key properties	Layer
Zone	h3, res, centroid (point), district	Shared
Sensor	deviceId, location, first_ts, last_ts, quality	Operational
ZoneReading	h3, ts, pm25, source (sensor, simulated, model), confidence, n_sensors	Operational
Hub	id, name, location	Operational
Rider	id, name, status, location, exposure_budget, exposure_used, activity_factor	Operational
Order	id, pickup, drop, window_start, window_end, status	Operational
Route	id, provider, duration_s, distance_m, polyline, dose, margin_s, coverage, rank, selected	Operational
Alert	id, type, severity, ts, status	Operational
PushSub	endpoint, keys, created_at	Operational

Label	Key properties	Layer
DemoClock	now_ts, speed, running	Operational
Place	id, name, type (landmark, metro, market, sector), location, city	Knowledge
Alias	text (normalized), votes	Knowledge
GRAPStage	level, name, active	Knowledge
Rule	id, description, source_url, effective_from	Knowledge
VehicleType	name, exposure_factor	Knowledge
MaskType	name, filtration	Knowledge

Relationships

Relationship	Direction	Properties	Purpose
IN_ZONE	Sensor or Place to Zone	none	Spatial assignment
OF	ZoneReading to Zone	none	Reading belongs to a zone
DERIVED_FROM	ZoneReading to Sensor	weight	Traceable explanations
ADJACENT	Zone to Zone	none	Neighbor search, avoidance routing
HAS_CANDIDATE	Order to Route	none	Alternatives for an order
PASSES_THROUGH	Route to Zone	idx, seconds, bucket	Ordered zone list with arrival bucket
ASSIGNED	Rider to Order	ts	Current assignment
EXPOSED_IN	Rider to Zone	ts, dose	Exposure ledger
ABOUT	Alert to Order or Rider	none	What the alert concerns

Relationship	Direction	Properties	Purpose
REFERS_TO	Alias to Place	votes , confidence	Place resolution cache
TRIGGERS , RESTRICTS , APPLIES_TO	GRAPStage to Rule to VehicleType or Zone	none	Policy reasoning
USES , WEARS , HAS_SUBSCRIPTION	Rider to VehicleType, MaskType, PushSub	none	Rider attributes

Constraints and indexes

```

CREATE CONSTRAINT zone_h3 IF NOT EXISTS FOR (z:Zone) REQUIRE z.h3 IS UNIQUE;
CREATE CONSTRAINT sensor_id IF NOT EXISTS FOR (s:Sensor) REQUIRE s.deviceId
IS UNIQUE;
CREATE CONSTRAINT rider_id IF NOT EXISTS FOR (r:Rider) REQUIRE r.id IS
UNIQUE;
CREATE CONSTRAINT order_id IF NOT EXISTS FOR (o:Order) REQUIRE o.id IS
UNIQUE;
CREATE CONSTRAINT route_id IF NOT EXISTS FOR (r:Route) REQUIRE r.id IS
UNIQUE;
CREATE CONSTRAINT alias_text IF NOT EXISTS FOR (a:Alias) REQUIRE a.text IS
UNIQUE;
CREATE CONSTRAINT place_id IF NOT EXISTS FOR (p:Place) REQUIRE p.id IS
UNIQUE;
CREATE INDEX zone_reading_lookup IF NOT EXISTS FOR (zr:ZoneReading) ON
(zr.h3, zr.ts);
CREATE POINT INDEX place_location IF NOT EXISTS FOR (p:Place) ON
(p.location);

```

Key queries

Route dose and coverage, using the arrival bucket stored on each **PASSES_THROUGH** (activity and mask factors are applied in the service layer):

```

MATCH (o:Order {id: $orderId})-[:HAS_CANDIDATE]->(r:Route)-
[p:PASSES_THROUGH]->(z:Zone)
OPTIONAL MATCH (zr:ZoneReading {h3: z.h3, ts: p.bucket})
RETURN r.id AS route,
       sum(coalesce(zr.pm25, 0) * p.seconds / 3600.0) AS dose_base,
       toFloat(count(zr)) / count(p) AS coverage
ORDER BY dose_base

```

Candidate riders: budget left, vehicle not restricted on this route by an active GRAP rule, least exposed first:

```
MATCH (rd:Rider {status: 'available'})-[:USES]->(v:VehicleType)
WHERE rd.exposure_used < 0.8 * rd.exposure_budget
  AND NOT EXISTS {
    MATCH (:GRAPStage {active: true})-[:TRIGGERS]->(ru:Rule)-[:RESTRICTS]->
(v)
    MATCH (ru)-[:APPLIES_TO]->(:Zone)<-[:PASSES_THROUGH]-(:Route {id:
$routeId})
  }
RETURN rd.id, rd.exposure_used, rd.exposure_budget
ORDER BY rd.exposure_used ASC
LIMIT 10
```

Explanation subgraph passed to Gemini, so every number in the sentence comes from the graph:

```
MATCH (r:Route {id: $routeId})-[p:PASSES_THROUGH]->(z:Zone)
MATCH (zr:ZoneReading {h3: z.h3, ts: p.bucket})
OPTIONAL MATCH (zr)-[:DERIVED_FROM]->(s:Sensor)
OPTIONAL MATCH (:GRAPStage {active: true})-[:TRIGGERS]->(ru:Rule)-
[:APPLIES_TO]->(z)
RETURN z.h3 AS zone, p.seconds AS seconds, zr.pm25 AS pm25, zr.source AS
source,
      collect(DISTINCT s.deviceId) AS sensors, collect(DISTINCT
ru.description) AS rules
ORDER BY p.idx
```

Check the Cypher against your Neo4j version before relying on it; the syntax follows Neo4j 5.

13. App information architecture

The product has two apps: an eight-screen dispatcher console in Streamlit and a three-screen rider app served by FastAPI. Both follow the same replay clock.

flowchart LR

```

P[Product] --> DC[Dispatcher console<br/>Streamlit]
P --> RA[Rider app<br/>FastAPI + PWA]
DC --> OP[Operate<br/>3 screens]
DC --> MO[Monitor<br/>2 screens]
DC --> CF[Configure and review<br/>3 screens]
RA --> R1[Today]
RA --> R2[Order detail]
RA --> R3[Alerts and settings]

```

Dispatcher console

Group	Screen	Purpose	Main data	Primary actions
Operate	Console (home)	Live overview of the fleet under the replay clock	Riders, Orders, Zones, ZoneReadings, Alerts	Play, pause, speed, jump; inject spike; open order; acknowledge alert
Operate	Route planner	Search places, compare candidate routes, recommend	Place, Alias, Route, Zone	Search; pick departure and window; choose route; assign
Operate	Orders and fleet	Manage orders, riders, budgets and assignment	Order, Rider, Route	Create simulated order; assign or reassign; see why a rider is ineligible
Monitor	Zones and AQI	Hex heatmap over time with confidence and sensor layers	Zone, ZoneReading, Sensor	Move time slider; toggle layers; inspect a zone
Monitor	Alerts	Full alert log	Alert	Filter; acknowledge; jump to the related order
Configure and review	Policy	Set the active GRAP stage and see rules per vehicle type	GRAPStage, Rule, VehicleType	Change stage; open rule source

Group	Screen	Purpose	Main data	Primary actions
Configure and review	Reports	Exposure by rider, fairness, dose avoided vs fastest	Rider, EXPOSED_IN, Route	Choose period; compare with baseline
Configure and review	Data and method	Sources, cleaning rules, formulas, limits, replay disclosure	Pipeline stats	Read only

Rider app

Screen	Purpose	Main data	Primary actions
Today	Assigned orders in delivery order	Order, Route	Open an order
Order detail	Route on a map, ETA and window, exposure for this order, reroute prompt	Order, Route, Zone, Alert	Accept reroute; mark delivered
Alerts and settings	Alert history and push permission	Alert, PushSub	Enable or disable push

The exposure meter (used vs budget) is a persistent header on every rider screen.

Global elements on every screen

- Replay clock bar showing the CSV timestamp being replayed, with a "Replay" label.
- PM2.5 color legend using the severity bands in section 6.
- Confidence badge on any score, and a "Simulated" badge on injected spikes.
- Link to Data and method.

Route comparison layout (top to bottom)

1. Recommendation banner: dose change in percent, extra minutes, window met or at risk.
2. Route cards side by side: duration, dose, window margin, coverage, confidence.
3. Google map with routes and zone colors.
4. One-sentence explanation from the grounded subgraph.
5. Expandable zone breakdown: zone, seconds, PM2.5, source, sensors.

Navigation and URLs

- Streamlit sidebar lists the eight screens in the group order above.
- Deep links: an alert opens its order in Orders and fleet; a route card opens its zone breakdown.
- Rider app paths: `/rider` , `/rider/order/{id}` , `/rider/settings` . The service worker file is served at `/sw.js` .
- Each Streamlit screen is one file under `pages/` , prefixed with its order number.

14. Service modules and APIs

The core package is split by responsibility so each module can be built, tested and demoed on its own. Neither UI contains scoring logic.

Repository layout

```
core/
  places/      resolve.py (alias, Gemini, geocode, candidates)
  routing/     google_routes.py, polyline_zones.py, scoring.py, recommend.py
  exposure/    dose.py, severity.py
  dispatch/    candidates.py, assign.py, fairness.py
  kg/          aliases.py, policy.py, explain.py
  alerts/      evaluator.py, push.py
  clock/       demo_clock.py
  db/          client.py, queries/
  etl/         profile.py, clean.py, aggregate.py, zones.py, load.py, seed.py
  api/         FastAPI app, rider app static files, sw.js
  ui/          Streamlit entry and pages/
  tests/
```

Module responsibilities

Module	Responsibility	Depends on
<code>places</code>	Turn a raw query into a confirmed place with coordinates and zone	<code>kg</code> , Gemini, Google Geocoding, Neo4j
<code>routing</code>	Fetch alternatives, map polylines to zones with arrival buckets, score, recommend	Google Routes, <code>exposure</code> , Neo4j

Module	Responsibility	Depends on
exposure	Dose formula, severity bands, coverage calculation	none (pure functions)
dispatch	Candidate riders, assignment, fairness ordering	routing , kg , Neo4j
kg	Alias graph, GRAP rules, grounded explanations	Neo4j, Gemini
alerts	Evaluate triggers per tick, store alerts, send push	clock , routing , Neo4j
clock	Replay time, speed, jump; spike injection	Neo4j

External APIs

API	Used for	Handling
Google Geocoding (or Places text search)	Normalized string to coordinates	Bound to the NCR area; cache the result on the Place and Alias nodes; server-side key
Google Routes API	Traffic-aware alternatives with duration, distance, polyline	Request only needed fields; store derived zone lists and scores, not raw geometry beyond the session; re-check the current terms
Google Maps JavaScript API	Display routes and zone overlays	Key restricted by HTTP referrer
Gemini (Flash-class model)	Place normalization; one-sentence explanations	JSON output, low temperature, short timeout; on failure send the raw query to the geocoder

Gemini contract: place normalizer

```
{
  "query": "Connaught Place, New Delhi",
  "candidates": ["Connaught Place, New Delhi", "Rajiv Chowk metro station, New Delhi"],
  "confidence": 0.93,
  "ambiguous": false,
  "city_hint": "Delhi"
}
```

The system prompt tells the model to expand local abbreviations (CP means Connaught Place, GK means Greater Kailash), never invent places, and return only JSON. The geocoder result always overrides the model when they disagree; the user picks when confidence is below 0.7.

Gemini contract: explainer

- Input: the subgraph rows from the explanation query, plus the recommendation deltas.
- Instruction: write one sentence for a dispatcher using only the facts provided; add no numbers.
- Guard: every number in the output must appear in the input; otherwise use a templated sentence instead.

Internal REST endpoints (FastAPI)

Method and path	Purpose
POST /api/places/resolve	Query in, place candidates out
POST /api/routes/plan	Start, destination, departure, window in; scored routes and recommendation out
POST /api/orders/{id}/assign	Assign a rider; returns eligibility reasons on failure
GET /api/zones?ts=	Zone readings for a replay time
POST /api/demo/clock	Set speed, pause, jump
POST /api/demo/spike	Inject a simulated PM2.5 spike into a zone
GET /api/alerts	Alert list with filters
POST /api/alerts/{id}/ack	Acknowledge an alert
POST /api/push/subscribe	Store a rider’s push subscription

Streamlit calls the core package directly; the REST layer serves the rider app and scripted tests.

15. Deployment and build order

Deploy two Render web services and one external Neo4j AuraDB instance, and build in the order that gets the MVP loop working first. Confirm current Render and Aura plan limits before relying on them.

Deployment

Piece	Where	Start command or note
Dispatcher console	Render web service	<code>streamlit run ui/app.py --server.port \$PORT --server.address 0.0.0.0</code>
API, rider app, push	Render web service	<code>uvicorn api.main:app --host 0.0.0.0 --port \$PORT</code>
Database	Neo4j AuraDB	Load aggregates from a local ETL run
Secrets	Render environment variables	Never committed; browser Maps key restricted by referrer

The replay clock is derived, not ticked: demo time is computed from an anchor timestamp, the speed and the wall clock, so it stays correct after a free-tier service sleeps. The alert evaluator runs on an interval while the API is awake and once on every dashboard refresh.

Environment variables

Variable	Used by
<code>NEO4J_URI</code> , <code>NEO4J_USER</code> , <code>NEO4J_PASSWORD</code>	Both services, ETL
<code>GOOGLE_MAPS_SERVER_KEY</code>	Geocoding and Routes calls
<code>GOOGLE_MAPS_BROWSER_KEY</code>	Maps JavaScript API in the UIs
<code>GEMINI_API_KEY</code>	Place normalizer and explainer
<code>VAPID_PUBLIC_KEY</code> , <code>VAPID_PRIVATE_KEY</code> , <code>VAPID_SUBJECT</code>	Web push

Variable	Used by
DEMO_AREA_BOUNDS , H3_RES , SENSOR_RADIUS_KM	Geocoder bounds, zoning, interpolation

Build order

Phase	Deliverable	Done when
0	Profile the CSVs; choose the file subset and demo area	Profile report answers the open questions on coverage and time range
1	ETL: clean, aggregate, zone, load	Zone readings queryable in Neo4j for the replay window
2	Scoring core with hardcoded routes	Dose, margin and coverage pass unit tests
3	Google routes, map display, route comparison screen	MVP loop works for one origin and destination
4	Place search: alias seed, Gemini, geocoder, picker	50-query labeled set scored
5	Replay clock, spike injector, in-app alerts	Spike changes route recommendation live
6	Grounded explanations	Number-check guard passes on every output
7	Rider app and web push	Push arrives on a real device or laptop browser
8	P1: budgets, dispatch, fairness, GRAP rules, avoidance routes, reports	Fairness beats nearest-rider baseline in simulation
9	Demo hardening: fixed scenario, wake-up routine, recorded backup video, final metrics	Full run-through without manual fixes

Demo flow

1. Zones and AQI at a calm time, then jump to the pollution rise.

2. Search "cp"; show the resolved place and the route comparison with the headline dose, time and window result.
3. Inject a spike on the active route; show the alert, the rider push notification and the reroute.
4. Raise the GRAP stage; show the candidate list change.
5. Open Reports; show exposure spread against the nearest-rider baseline.